1. Endpoint가 정확히 이해가 안 돼서 추가적으로 조사해 보았다.

엔드포인트는 보통 **[HTTP Method] + [URI(경로)]**의 조합으로 완성된다.

> **POST https://umc.com/users/login**

- **HTTP Method (POST):** "어떤 행위를 할 것인가?" (로그인 정보를 '제출'한다)
- **Base URL (https://umc.com):** "어느 서버로 갈 것인가?" (UMC라는 서비스의 서버)
- **Path (/users/login):** "그 서버 안의 어느 방(입구)으로 갈 것인가?" (사용자 관련 기능 중 로그인 방)

이 세 가지가 합쳐져서 하나의 **엔드포인트**가 되는 것이다.

2. Node.js 와 Express 각각의 역할에 대해 알아보았다

Node.js는 자바스크립트를 실행할 수 있는 환경이고, Express는 그 위에서 웹 서버를 쉽게 만들게 해주는 도구(프레임워크)다.

---

1. Node.js
Node.js는 자바스크립트를 브라우저(Chrome 등) 밖에서도 실행할 수 있게 해주는 **런타임(Runtime)**이다.

- **역할:** 컴퓨터에서 자바스크립트 코드를 읽고 해석해서 실행하는 환경을 제공한다.
- **특징:** Node.js 자체만으로도 웹 서버를 만들 수는 있지만, 코드가 매우 길고 복잡하며 설정할 것이 너무 많다.

2. Express.js
Express는 Node.js 위에서 동작하는 가장 인기 있는 **웹 프레임워크**다.

- **역할:** 웹 서버를 만들기 위해 반복적으로 작성해야 하는 복잡한 코드들을 미리 짜놓은 라이브러리다.
- **특징:** "라우팅(URL 관리)", "미들웨어(중간 처리)", "에러 핸들링" 등을 아주 짧고 직관적인 코드로 작성할 수 있게 해준다.

항목	Node.js	Express.js
정체	런타임 환경 (Runtime)	웹 프레임워크 (Framework)
비유	자동차의 **엔진**	자동차의 **차체와 편의 장치**

관계	기반 인프라	Node.js 위에서 실행됨
주요 기능	파일 시스템 접근, 네트워크 통신 등	라우팅, HTTP 요청/응답 처리

4. **코드예시**

Pure Node.js (생코딩)

```
const http = require('http');
const server = http.createServer((req, res) => {
  if (req.url === '/') {
    res.writeHead(200, { 'Content-Type': 'text/plai
n' });
    res.end('Hello World');
  }
});
server.listen(3000);
```

Express 사용

```
const express = require('express');
const app = express();

app.get('/', (req, res) => {
  res.send('Hello World');
});

app.listen(3000);
```

Node.js만 쓰면 요청(Request)과 응답(Response)의 상세한 부분까지 일일이 제어해야 하지만, Express를 쓰면 **이 주소(`/`)로 GET 요청이 들어오면 이 메시지를 보내라**는 핵심 로직만 간결하게 짤 수 있다.